# Trabalho 01

**Análise de complexidade em estruturas de listas**

Gabriel Leite Fonseca e César Augusto Bispo Santos

29/03/2026

---

## Seção 01

## Seção 02

### 2 - Análise Matemática de complexidade dos Algoritmos

#### 2.1 - Quantidade de Nós

```
public int quantidadeNos() {
    return this.qtd;
}
```

A forma optada por se implementar o método responsável por retornar a quantidade de nós foi a mais simples, tendo um atributo de quantos nós existe dentro da classe e apenas retornando esse atributo. O método é $\mathbf{O(1)}$ por ser constante, independente da quantidade de nós sempre retornando a mesma no mesmo tempo.

#### 2.2 - Adicionar

```
public void adicionar(T novoValor) {
    Node<T> novo_ultimo  = new Node<T>();
    novo_ultimo.value = novoValor;
```

```
    if (this.first == null) { this.first = this.last = novo_ultimo; }
    else {
        this.last.next    = novo_ultimo;
        this.last         = novo_ultimo;
        this.last.next    = null;
    }
    this.qtd += 1;
}
```

A complexidade do método **adicionar** é **O(1)**, pois é realizada uma inserção diretamente no final da lista, independente de quantos elemento a lista possua e sem precisar percorre-los um a um. Isso é possível pela referência apontada por *last* e pela natureza de adicionar sem se preocupar com a posição em que o elemento será inserido.

A título de comparação, em um método de inserção no qual o elemento deve ser adicionado em uma posição específica definida, por exemplo, por um índice informado pelo usuário, a complexidade de tempo passa a ser O(n).

Isso ocorre porque é necessário percorrer a estrutura elemento por elemento até alcançar a posição desejada. Esse processo de busca é linear em relação ao número de elementos da lista. Após localizar a posição correta, as demais operações de inserção possuem custo constante, mas não alteram a complexidade total do método, que permanece O(n) devido ao custo dominante.

### 2.3 - Pesquisar

```
41      public T pesquisar(T valor) {
42          No<T> noAtual = this.primeiro; C1
43
44          while (noAtual != null) {
45              if (Objects.equals(noAtual.getValor(), valor)) {
46                  return noAtual.getValor();
47              }
48              noAtual = noAtual.getProx();
49          }
50          return null; C3
51      }
```

N VEZES

C2

Figure 1: Código método de pesquisa

> **Note**
>
> Para facilitar a análise, as linhas do código foram agrupadas em constantes (C1, C2, C3), representando operações de custo fixo (abordagem adotada para restante do relatório). O ponto principal está no laço de repetição, que pode executar até n vezes, dependendo do tamanho da lista. Por isso, o comportamento do algoritmo é determinado por esse trecho.

A complexidade do método pesquisar é **O(n)**, pois a busca é realizada de forma sequencial ao longo da lista encadeada. Para localizar um valor, o algoritmo percorre os nós um a um, comparando seus conteúdos até encontrar o elemento desejado ou atingir o final da estrutura.

No pior caso, quando o elemento não está presente (ou encontra-se na última posição), o método percorre todos $N$ elementos da lista até que o ponteiro atual se torne nulo, encerrando a busca. Nesse cenário, o número de operações cresce proporcionalmente ao tamanho da lista, caracterizando um crescimento linear.

### 2.4 - Remover

```
53      public boolean remover(T valor) {
54          No<T> noAtual    = this.primeiro; C1
55          No<T> noAnterior = null; C2
56
57          while (noAtual != null) {
58              if (Objects.equals(noAtual.getValor(), valor)) {
59                  if (noAnterior == null) {
60                      // remoção do primeiro nó
61                      this.primeiro = noAtual.getProx();
62                      if (this.primeiro == null) { this.ultimo = null; }
63                  } else if (noAtual.getProx() != null) {
64                      // remoção de nó do meio
65                      noAnterior.setProx(noAtual.getProx());
66                  } else {
67                      // remoção do último nó
68                      this.ultimo = noAnterior;
69                      noAnterior.setProx(null);
70                  }
71                  noAtual.setProx(null); // limpa referência do nó removido
72                  this.qtd--;
73                  return true;
74              }
75
76              noAnterior = noAtual;
77              noAtual    = noAtual.getProx();
78          }
79          return false; C4
80      }
81  }
```

C3

Figure 2: Código método de pesquisa

O método remover, assim como o método pesquisar, apresenta complexidade de tempo **O(n)**, pois depende de uma percurso sequencial da lista para localizar o elemento a ser removido.

Embora o corpo do laço contenha mais operações em comparação ao método de busca, essas operações possuem custo constante. Mesmo que existam diferenças de tempo entre elas, tais variações são desprezíveis na análise, podendo ser agrupadas em uma constante (C3).

No pior caso, quando o elemento não está presente ou encontra-se na última posição, o algoritmo percorre todos os **N** elementos da lista. Assim, o custo total pode ser representado como

proporcional a **N** e caracterizar um crescimento linear.

Dessa forma, a complexidade do método é **O(n)**.

# 3 - Análise ArrayList e LinkedList

## 3.1 - LinkedList

Diferente de um Array o qual deve ter um tamanho definido de memória, a estrutura **LinkedList** do Java apresenta alocação dinâmica, alterando o espaço armazenado a medida que elementos são adicionados ou removidos. A **LinkedList** faz parte da **Java Collections** e implementa a *AbstractSequentialList*, o que - de forma prática - implica que o acesso a um elemento da lista é feita de forma sequencial, percorrendo um a um.

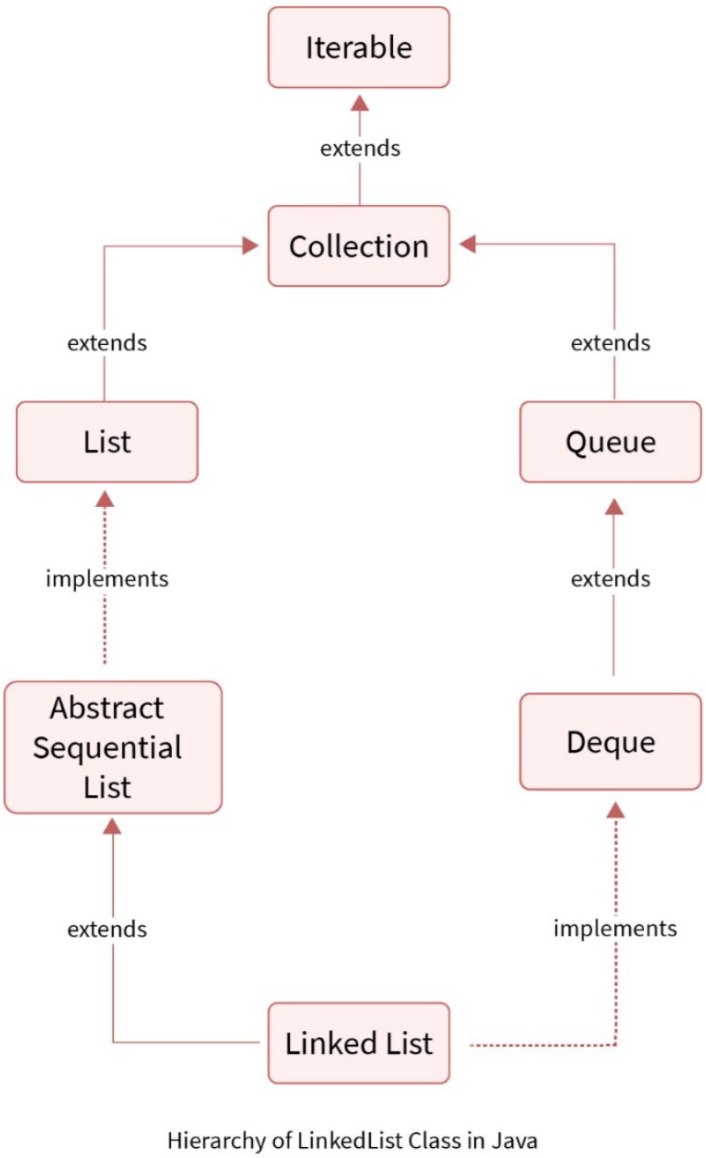


Figure 3: Implementação LinkedList

A estrutura mantém referências tanto para o primeiro quanto para o último nó da lista. Além disso, cada nó possui ponteiros para o elemento seguinte (next) e para o anterior (prev), caracterizando a estrutura como uma lista duplamente encadeada. Essa característica permite inserções eficientes nas extremidades, reduzindo o custo dessas operações para tempo constante.

```java
// Implemtentação: apontando para primeiro e último elementos

public class LinkedList<E>
    extends AbstractSequentialList<E>
    implements List<E>, Deque<E>, Cloneable, java.io.Serializable
{
    transient int size = 0;

    /**
     * Pointer to first node.
     */
    transient Node<E> first;

    /**
     * Pointer to last node.
     */
    transient Node<E> last;
```

### Métodos

#### Adicionar

A forma de se adicionar a um elemento em uma **LinkedList** é feito - principalmente - pelo método **add(E e)**. Além dele, também existem variações como **addFirst(E e)**, **addLast(E e)** e **add(int index, E element)**, cada uma com comportamentos distintos. Por existir ponteiros apontando para o primeiro e último elemento, as funções **addFirst** e **addLast** tem uma complexidade constante, por não serem alteradas de forma correlacionada a quantidade de elementos na lista.

O método **add(E e)** delega sua execução ao método **linkLast(E e)**, responsável por inserir o novo elemento ao final da lista utilizando a referência ao último nó. Como não há necessidade de percorrer a estrutura, a operação possui complexidade O(1).

```java
// Método de adicionar elemento a lista encadeada
public boolean add(E e) {
    linkLast(e);
    return true;
}

// Método chamado por add
void linkLast(E e) {
    final Node<E> l = last;
```

```
    final Node<E> newNode = new Node<>(l, e, null);
    last = newNode;
    if (l == null)
        first = newNode;
    else
        l.next = newNode;
    size++;
    modCount++;
}
```

Contudo, o método **add(int index, E element)** - diferente dos citados anteriormente - acaba por ter complexidade Linear. Ao receber o índice como parâmetro, a lista precisa percorrer seus elementos nó a nó até alcançar a posição correspondente. Esse processo de localização possui custo linear em relação ao número de elementos armazenados.

O pior caso ocorre quando o índice está aproximadamente no meio da lista, pois - mesmo com a otimização que permite iniciar o percurso a partir do início ou do fim - ainda será necessário percorrer n/2 elementos. Assim, o tempo de execução cresce proporcionalmente ao tamanho da lista, caracterizando a complexidade **O(n)**.

### Remover

A remoção de elementos em uma **LinkedList** pode ser realizada de duas formas principais: por índice ou valor sendo - para ambos os casos - complexidade **O(N)**.

Para a remoção por objeto, cada nó é comparado com o valor passado como parâmetro, em que toda a estrutura será percorrida no caso do elemento não estar presente, fazendo com que o tempo de execução cresça proporcionalmente ao número de elementos da lista.

A implementação também trata explicitamente o caso em que o valor buscado é nulo, decorrido o fato da **LinkedList** permitir a inserção de elementos **null**. Dessa forma, a verificação inicial evita a chamada do método equals sobre uma referência nula, o que resultaria em uma exceção.

```
public boolean remove(Object o) {
    if (o == null) {
        for (Node<E> x = first; x != null; x = x.next) {
            if (x.item == null) {
                unlink(x);
                return true;
            }
        }
    } else {
```

```
        for (Node<E> x = first; x != null; x = x.next) {
            if (o.equals(x.item)) {
                unlink(x);
                return true;
            }
        }
    }
    return false;
}
```

Já a remoção por índice pode ter um tempo de execução *menor* ao pior caso da remoção por objeto por percorrer apenas metade da lista, mas também cresce o tempo de execução de forma proporcional a quantidade de elementos na lista. caso o índice esteja na primeira metade da lista, a busca é realizada a partir do início (first); caso contrário, inicia-se a partir do final (last). Essa estratégia reduz a quantidade de elementos percorridos, limitando o percurso a - no máximo - *n/2* elementos. Apesar disso, sua complexidade se mantém **O(N)**.

```
// Remover por índice
public E remove(int index) {
    checkElementIndex(index);
    return unlink(node(index));
}

Node<E> node(int index) {
    // assert isElementIndex(index);

    if (index < (size >> 1)) {
        Node<E> x = first;
        for (int i = 0; i < index; i++)
            x = x.next;
        return x;
    } else {
        Node<E> x = last;
        for (int i = size - 1; i > index; i--)
            x = x.prev;
        return x;
    }
}
```

### Busca

A busca em uma LinkedList pode ser realizada de duas formas principais: por índice ou por valor do elemento.

Na busca por objeto **contains(Object O)**, é utilizada internamente o método **indexOf(Object O)**, sendo parcialmente igual ao método de remoção por índice mas - ao invés de fazer uso do **unlink** e retornar um booleano, o método é responsável por retornar o índice no qual o objeto pertence. Por se tratar de um algoritmo que pecorrerá todos os elementos da lista em seu pior caso e crescer o tempo de execução dado o tamanho da lista, podemos dizer que o método é **O(N)**

```
public int indexOf(Object o) {
    int index = 0;
    if (o == null) {
        for (Node<E> x = first; x != null; x = x.next) {
            if (x.item == null)
                return index;
            index++;
        }
    } else {
        for (Node<E> x = first; x != null; x = x.next) {
            if (o.equals(x.item))
                return index;
            index++;
        }
    }
    return -1;
}
```

Na busca por índice, realizada pelo método **get(int index)**, o funcionamento é semelhante ao da remoção por índice no que diz respeito à localização do elemento. Em ambos os casos, é utilizado o método **node(index)**, responsável por percorrer a lista até alcançar a posição desejada. A principal diferença está na operação final em que, enquanto o método de remoção por meio do método unlink, o método get apenas acessa o valor armazenado no nó, por meio do atributo item, e o retorna.

```
public E get(int index) {
    checkElementIndex(index);
    return node(index).item;
}
```

## 3.2 - ArrayList

O **ArrayList** — assim como o **LinkedList** — faz parte do **Java Collections Framework**, sendo uma implementação baseada em array dinâmico.

Diferentemente de arrays tradicionais, o **ArrayList** possui redimensionamento automático, permitindo que sua capacidade aumente conforme novos elementos são adicionados.

Esse crescimento é realizado internamente por meio do método **grow()**, que é acionado quando a capacidade atual do array é insuficiente para armazenar novos elementos.

```
private Object[] grow(int minCapacity) {
    int oldCapacity = elementData.length;
    if (oldCapacity > 0 || elementData != DEFAULTCAPACITY_EMPTY_ELEMENTDATA) {
        int newCapacity = ArraysSupport.newLength(oldCapacity,
                minCapacity - oldCapacity, /* minimum growth */
                oldCapacity >> 1           /* preferred growth */);
        return elementData = Arrays.copyOf(elementData, newCapacity);
    } else {
        return elementData = new Object[Math.max(DEFAULT_CAPACITY, minCapacity)];
    }
}
```

### Adicionar

Ao analisar o método **add(E e)** do **ArrayList**, estamos tratando de uma inserção no final da lista, ou seja, o elemento é adicionado diretamente na posição correspondente ao valor atual de size, seguida da sua incrementação. No cenário mais comum essa operação ocorre de forma direta no array interno, sem necessidade aumento do array e tendo tempo constante.

Entretanto, no pior caso, quando a capacidade do array já foi totalmente utilizada, é necessário redimensionar o array por meio do método **grow()**. Nesse processo, um novo array maior é criado e todos os elementos do array antigo são copiados para esse novo array.

```
private Object[] grow(int minCapacity) {
    int oldCapacity = elementData.length;
    if (oldCapacity > 0 || elementData != DEFAULTCAPACITY_EMPTY_ELEMENTDATA) {
        int newCapacity = ArraysSupport.newLength(oldCapacity,
                minCapacity - oldCapacity, /* minimum growth */
                oldCapacity >> 1           /* preferred growth */);
        return elementData = Arrays.copyOf(elementData, newCapacity);
    } else {
        return elementData = new Object[Math.max(DEFAULT_CAPACITY, minCapacity)];
```

```
}
}
```

Isso acaba por aumentar o tempo de execução dada a quantidade de elementos da lista, levando o método a ter complexidade linear (**(O(n))**).

### Remover

Quando falamos dos métodos de remoção, temos uma remoção por índice e uma por objeto, ambas tendo complexidade linear. O método de remoção por índice **(remove(int index))** remove diretamente o elemento é acessado a partir do índice informado em tempo constante, mas - ao chamar o **fastRemove** - o método **System.arraycopy** é chamado, o qual acaba por deslocar todos os elementos à direita do índice removido uma posição para a esquerda, não deixando lacunas. Por conta disso, ao analisar o pior cenário - que seria remover o primeiro elemento e ter deslocar os elementos a da esquerda, percorrendo um a um - sua complexidade aumenta dada a quantidade de elementos na lista para o pior caso, sendo considerada **O(n)**.-

```
// Remoção por índice
public E remove(int index) {
    Objects.checkIndex(index, size);
    final Object[] es = elementData;

    @SuppressWarnings("unchecked") E oldValue = (E) es[index];
    fastRemove(es, index);

    return oldValue;
}

// Método chamado na remoção por índice
private void fastRemove(Object[] es, int i) {
    modCount++;
    final int newSize;
    if ((newSize = size - 1) > i)
        System.arraycopy(es, i + 1, es, i, newSize - i);
    es[size = newSize] = null;
}
```

Já na remoção por elemento, o método percorre elemento a elemento até encontrar o valor passado como parâmetro, o que, no pior caso, quando o elemento está na última posição da lista, exige percorrer todos os elementos, resultando em complexidade **O(n)** proporcional à quantidade de elementos armazenados. Contudo, mesmo no caso em que o elemento está

no início da lista, após a remoção o método **System.arraycopy** é chamado, assim como na remoção por índice, o que provoca o deslocamento de todos os elementos à direita uma posição para a esquerda. Dessa forma, também há um custo linear nesse cenário, mantendo a complexidade **O(n)**.

```
public boolean remove(Object o) {
    final Object[] es = elementData;
    final int size = this.size;
    int i = 0;
    found: {
        if (o == null) {
            for (; i < size; i++)
                if (es[i] == null)
                    break found;
        } else {
            for (; i < size; i++)
                if (o.equals(es[i]))
                    break found;
        }
        return false;
    }
    fastRemove(es, i);
    return true;
}
```

### Busca

Os métodos de busca para o **ArrayList** podem ser por índice - sendo esse o de melhor desemepenho - e por objeto, sendo pior para o pior caso. Na busca por índice **get(int index)**, é passado um parâmetro para o método como parâmetro e o elemento é acessado pelo índice diretamente, levando o método a ser complexidade constante O(1).

```
public E get(int index) {
    Objects.checkIndex(index, size);
    checkForComodification();
    return root.elementData(offset + index);
}
```

Contudo, ao analisarmos a busca por elemento por meio do método **contains**, observamos que o método percorre elemento a elemento até encontrar o valor desejado ou até atingir o final da lista. No pior caso, quando o elemento não está presente ou se encontra na última posição, é necessário percorrer todo o array, fazendo com que o tempo de execução seja proporcional à

quantidade de elementos armazenados. Dessa forma, podemos caracterizar a operação como uma busca linear, com complexidade **O(n)**.

```
public boolean contains(Object o) {
    return indexOf(o) >= 0;
}

public int indexOf(Object o) {
    return indexOfRange(o, 0, size);
}

int indexOfRange(Object o, int start, int end) {
    Object[] es = elementData;
    if (o == null) {
        for (int i = start; i < end; i++) {
            if (es[i] == null) {
                return i;
            }
        }
    } else {
        for (int i = start; i < end; i++) {
            if (o.equals(es[i])) {
                return i;
            }
        }
    }
    return -1;
}
```